

◆ Gemini

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

def get_dark_channel(img, patch_size=15):
    """
    Computes the dark channel of an image.
    The dark channel is the minimum intensity in a local patch for each color
    """
    M, N, _ = img.shape
    padded = np.pad(img, ((patch_size // 2, patch_size // 2), (patch_size // 2, patch_size // 2)), mode='constant')
    dark_channel = np.zeros((M, N), dtype=np.float32)
    for i, j in np.ndindex(M, N):
        dark_channel[i, j] = np.min(padded[i:i + patch_size, j:j + patch_size, :], axis=-1)
    return dark_channel

def estimate_atmospheric_light(img, dark_channel, top_percent=0.1):
    """
    Estimates the atmospheric light by picking the brightest pixels in the dark channel
    """
    num_pixels = img.shape[0] * img.shape[1]
    num_top_pixels = int(num_pixels * top_percent)
    flat_dark_channel = dark_channel.flatten()
    flat_img = img.reshape(num_pixels, 3)

    # Get indices of the top 'num_top_pixels' in the dark channel
    indices = np.argsort(flat_dark_channel)[::-1][:num_top_pixels]

    # Return the average intensity of these pixels in the original image
    atmospheric_light = np.mean(flat_img[indices], axis=0)
    return atmospheric_light

def estimate_transmission_map(img, atmospheric_light, omega=0.95, patch_size=15):
    """
    Estimates the transmission map.
    """
    img_normalized = img / atmospheric_light # Element-wise division
    dark_channel_normalized = get_dark_channel(img_normalized, patch_size)
    transmission_map = 1 - omega * dark_channel_normalized
    return transmission_map

def dehaze(img, patch_size=15, top_percent=0.1, omega=0.95, t0=0.1):
    """
    Performs image dehazing using the Dark Channel Prior.
    """
    img_float = img.astype(np.float32) / 255.0 # Normalize image to [0, 1]

    # 1. Compute Dark Channel
```

```

dark_channel = get_dark_channel(img_float, patch_size)

# 2. Estimate Atmospheric Light
atmospheric_light = estimate_atmospheric_light(img_float, dark_channel, t0)

# 3. Estimate Transmission Map
transmission_map = estimate_transmission_map(img_float, atmospheric_light, t0)

# Clamp transmission map to a minimum value to avoid division by zero and
transmission_map = np.maximum(transmission_map, t0)

# 4. Recover the Scene Radiance (De-hazed Image)
#  $J = ((I - A) / \max(t, t_0)) + A$ 
dehazed_img = np.zeros_like(img_float)
for c in range(3):
    dehazed_img[:, :, c] = ((img_float[:, :, c] - atmospheric_light[c]) /
                             (transmission_map + t0)) + atmospheric_light[c]

dehazed_img = np.clip(dehazed_img * 255.0, 0, 255).astype(np.uint8)
return dehazed_img, transmission_map, atmospheric_light

# --- Example Usage ---

# Create a dummy hazy image for demonstration purposes
# In a real scenario, you would load an image:
# hazy_image = cv2.imread('path/to/your/hazy_image.jpg')
# hazy_image = cv2.cvtColor(hazy_image, cv2.COLOR_BGR2RGB) # Convert to RGB if needed

# Let's create a synthetic hazy image for a reproducible example
# A simple way to simulate haze is to blend the original image with a constant
# atmospheric light color.

def create_synthetic_haze(image, air_light=(0.7, 0.7, 0.7), beta=0.08): # beta is transmission
    img_float = image.astype(np.float32) / 255.0
    rows, cols, _ = image.shape
    depth_map = np.ones((rows, cols)) # Simplified: assume uniform depth

    # Transmission map  $t(x) = \exp(-\beta * d(x))$ 
    transmission = np.exp(-beta * depth_map)

    # Hazy image  $I(x) = J(x) * t(x) + A * (1 - t(x))$ 
    hazy_img = np.zeros_like(img_float)
    for c in range(3):
        hazy_img[:, :, c] = img_float[:, :, c] * transmission + air_light[c] * (1 - transmission)

    return np.clip(hazy_img * 255.0, 0, 255).astype(np.uint8)

# Create a simple non-hazy image (e.g., solid colors, or load a small image)
original_img = np.zeros((100, 150, 3), dtype=np.uint8)
original_img[:50, :75] = [255, 0, 0] # Red
original_img[:50, 75:] = [0, 255, 0] # Green
original_img[50:, :75] = [0, 0, 255] # Blue
original_img[50:, 75:] = [255, 255, 0] # Yellow

```

```

original_img[50:, 50:] = [255, 255, 0] # yellow

# Simulate a hazy version of the image
hazy_image = create_synthetic_haze(original_img, air_light=(0.8, 0.8, 0.8), b

print("Dehazing process started...")
dehazed_image, transmission_map, atmospheric_light = dehaze(hazy_image)
print("Dehazing process finished.")

# Display results
plt.figure(figsize=(15, 5))

plt.subplot(1, 3, 1)
plt.imshow(hazy_image)
plt.title('Hazy Image (Synthetic)')
plt.axis('off')

plt.subplot(1, 3, 2)
plt.imshow(transmission_map, cmap='gray')
plt.title('Transmission Map')
plt.axis('off')

plt.subplot(1, 3, 3)
plt.imshow(dehazed_image)
plt.title('Dehazed Image')
plt.axis('off')

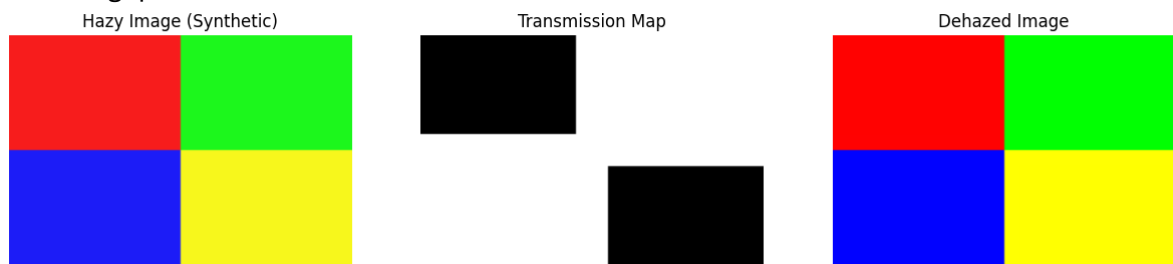
plt.show()

print(f"Estimated Atmospheric Light: {atmospheric_light}")

# Note: For real-world images, the Dark Channel Prior works best when
# there are enough dark regions in the image. Complex scenes or images
# with bright objects (like cars with headlights) in haze might require
# more advanced techniques or refinements like guided filtering.

```

Dehazing process started...
Dehazing process finished.



Estimated Atmospheric Light: [0.5415053 0.53177196 0.53692496]

